

# SIMATS ENGINEERING

## Department of Computer Science & Engineering ASSESSMENT TOOL 2 – DEBUGGING & OPTIMISATION TASK

|                      |                                |                      |                                                   |
|----------------------|--------------------------------|----------------------|---------------------------------------------------|
| <b>Course Code:</b>  | CSA12                          | <b>Course Title:</b> | Computer Architecture                             |
| <b>CO Assessed:</b>  | CO2                            | <b>Assessment:</b>   | Assessment Tool 2 – Debugging & Optimisation Task |
| <b>Weightage:</b>    | 20%                            | <b>Total Marks:</b>  | 25                                                |
| <b>Student Name:</b> | V Jeevanantham                 | <b>Reg. No.:</b>     | 192421234                                         |
| <b>Department:</b>   | Computer Science & Engineering | <b>Date:</b>         | 01-08-2026                                        |

### ANNEXURE A – ANSWERS

#### Question 1

A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.

#### Debugging the computation

In 8-bit two's complement, the representable signed range is -128 to +127. Converting the operands: +120 = 0111 1000 and +50 = 0011 0010. Adding these bit by bit gives:

```
0111 1000 (+120)
+ 0011 0010 (+50)
-----
1010 1010
```

The 8-bit result 1010 1010 has its most significant bit (MSB) set to 1, so it is interpreted as a negative number: taking two's complement gives -86. Two positive operands have produced a negative sum, which is impossible, so overflow has clearly occurred – the true sum (+170) simply cannot fit inside the 8-bit signed range.

#### Root cause

The bug is not in the addition logic itself (ripple addition of the bits is correct) but in the absence of an overflow check after the addition. The hardware/software silently truncates the carry out of the MSB and reinterprets the wrapped bit pattern as a valid signed value.

#### Proposed fix – overflow detection

Two equivalent and reliable overflow tests can be added immediately after the addition:

- 1. Carry-in / carry-out comparison rule:** Let  $C_{in}$  be the carry into the MSB position and  $C_{out}$  be the carry out of the MSB.  $Overflow = C_{in} \oplus C_{out}$ . In the example, the carry into bit 7 differs from the carry out of bit 7, so  $overflow = 1$ .
- 2. Sign-comparison rule:** Overflow occurs only when both operands have the same sign and the result's sign differs from theirs:  $Overflow = (Sign(A) == Sign(B)) \text{ AND } (Sign(A) != Sign(Result))$ . Here A and B are both positive (sign 0) but the result's sign bit is 1, so overflow is flagged.

Implementing either rule as a hardware overflow flag (as in an ALU's V flag) or as a software guard before storing the result lets the program detect the condition and respond – for example by promoting the operands to a wider data type (16-bit or 32-bit), by saturating the result at the maximum representable value, or by raising an exception/trap instead of silently returning a corrupted value.

## Question 2

*A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.*

### Debugging the bottleneck

In a ripple carry adder (RCA), each full-adder stage  $i$  can only produce its sum bit and its carry-out  $C_i$  once it has received the carry-in  $C_{i-1}$  from the previous stage. This creates a strictly serial dependency chain from bit 0 up to bit  $n-1$ . The worst case – and the case that dominates real-time performance – occurs when a carry generated at the least significant bit must ripple all the way through every intermediate stage to the most significant bit, for example when adding  $0000...01$  and  $1111...11$ . For an  $n$ -bit adder this gives a critical-path delay proportional to  $n$  gate delays ( $O(n)$ ), which is why wider registers (32-bit, 64-bit) cause a proportionally larger and more visible latency in a system with tight timing deadlines.

### Optimized solution – Carry Look-ahead Adder (CLA)

A carry look-ahead adder removes this serial dependency by computing every carry directly from the original inputs instead of waiting for the previous stage. For each bit position  $i$  it first defines two auxiliary signals in parallel:

```
Generate:  $G_i = A_i \cdot B_i$  (this stage produces a carry on its own)
Propagate:  $P_i = A_i \oplus B_i$  (this stage passes an incoming carry through)
```

Using these, each carry can be expanded algebraically without depending on the previous carry actually having been computed first, e.g.  $C_1 = G_0 + P_0 \cdot C_{in}$ ,  $C_2 = G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_{in}$ , and so on. Because every term in these expressions is available from the original  $A$  and  $B$  inputs, all carries can be produced by a two-level AND-OR network simultaneously rather than sequentially.

### Why it reduces delay

Since the carries are computed directly from  $G$  and  $P$  signals in parallel rather than rippling stage by stage, the critical path collapses from  $O(n)$  to essentially a constant number of gate delays (typically  $O(\log n)$  once look-ahead blocks are cascaded for wide words). This makes the CLA far better suited to a real-time system, at the cost of increased gate count/fan-in as word width grows – which in practice is managed by grouping bits into 4-bit look-ahead blocks and cascading a second level of look-ahead across the blocks (hierarchical/block CLA).

## Question 3

*A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.*

### Recap of the algorithm

Booth's algorithm multiplies signed numbers by scanning pairs of adjacent bits of the multiplier  $Q$ , together with an extra bit  $Q_{-1}$  initialised to 0, and deciding an action for the accumulator  $A$  based on the pair  $(Q_0, Q_{-1})$ :

```
00 → no operation, arithmetic shift right (ASR)
01 →  $A = A + M$ , then ASR
10 →  $A = A - M$ , then ASR
11 → no operation, ASR
```

This is repeated once per multiplier bit, and the combined  $(A, Q, Q_{-1})$  register is right-shifted arithmetically after every step.

## Debugging – likely sources of error

When the algorithm misbehaves specifically for negative operands (but works for positive ones), three implementation faults are the usual suspects:

1. **Logical instead of arithmetic shift:** if the shift step is implemented as a plain logical right shift, a 0 is always inserted into the vacated MSB. For a negative accumulator (MSB = 1) this destroys the sign, turning a negative partial product positive and corrupting every subsequent addition/subtraction.
2. **Incorrect  $Q_{-1}$  initialisation or update:**  $Q_{-1}$  must start at 0 and must receive the bit shifted out of  $Q_0$  at each step (it is part of the same shifted register). If it is left stale or reset incorrectly, the wrong bit pair is examined and the wrong operation (add instead of subtract, or vice versa) is triggered.
3. **Wrong bit-pair alignment / off-by-one scan:** comparing ( $Q_0$ ,  $Q_{-1}$ ) using the value of  $Q_{-1}$  from the wrong iteration (e.g. before rather than after the shift) shifts the whole decision table by one position, which shows up only when the multiplier or multiplicand is negative because that is when a sign-changing addition/subtraction is actually exercised.

## Corrections

The fix is to (a) always use an arithmetic shift right that replicates the current sign bit of A into the vacated MSB after each add/subtract, (b) initialise  $Q_{-1} = 0$  exactly once before the first iteration and update it as part of the same shift operation that shifts Q, and (c) verify, by tracing a small negative example by hand (e.g.  $M = -6$ ,  $Q = -3$  in 4-bit), that each of the n iterations reads the pair before the shift and applies the shift after the add/subtract. Adding this trace as a unit test with known negative operands and expected products will catch a regression in sign handling immediately.

## Question 4

*An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.*

## Debugging the workflow

The restoring division algorithm, for each of the n bits of the quotient, performs: (1) shift the remainder register left by one bit, (2) subtract the divisor from the partial remainder, (3) test the sign of the result – if it is non-negative, set the quotient bit to 1 and keep the subtracted value; if it is negative, set the quotient bit to 0 and, critically, add the divisor back ("restore") to undo the subtraction before moving to the next bit. In an embedded system dividing many operands, every bit position where the trial subtraction goes negative costs *two* full-width arithmetic operations (a subtract followed by a compensating add) instead of one, and this restoring add contributes nothing to the final answer – it exists purely to undo the previous step. Profiling the workflow shows execution time growing with the number of negative trial subtractions, which is exactly the wasted work.

## Optimized approach – non-restoring division

Non-restoring division removes the separate restore step entirely by letting the sign of the current remainder decide the *next* operation instead of undoing the current one. The revised per-bit workflow is:

```
If remainder is non-negative: shift left, then subtract divisor, set quotient bit 1.  
If remainder is negative: shift left, then add divisor (instead of subtracting), set quotient bit 0.  
After the final iteration, if the remainder is negative, a single corrective addition of the divisor restores the true remainder (only one restore for the whole operation, not one per bit).
```

## Why it is faster

Every iteration now performs exactly one add-or-subtract operation regardless of sign, compared to restoring division's occasional two-operation iterations. Over  $n$  bits this removes up to  $n$  redundant additions, and because the decision of add-vs-subtract only needs the sign bit of the previous remainder (already available), it also shortens the critical path per iteration, making non-restoring division a better fit for latency-sensitive embedded workloads.

## Question 5

*A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.*

### Debugging the accuracy loss

IEEE 754 single precision stores a value as 1 sign bit, 8 exponent bits, and 23 explicit mantissa bits (with an implicit leading 1). Before two floating-point numbers can be added, the hardware must align their exponents by right-shifting the mantissa of the number with the smaller exponent until both exponents match. When one operand is very large and the other is very small, this required shift can exceed the 23 available mantissa bits: the small operand's significant bits are shifted entirely out of the mantissa field and are lost (rounded to zero) before the addition even happens. The subsequent normalization and rounding step then simply returns the large operand unchanged, silently discarding the small value's contribution – this is the classic symptom reported (adding a tiny number appears to have "no effect").

### Where exactly the error is introduced

1. **Exponent alignment:** the shift amount equals the exponent difference; once it exceeds the mantissa width plus guard bits, all information from the smaller number is gone.
2. **Normalization:** after alignment the result is renormalized to restore the implicit leading 1 form, which cannot recover bits already shifted away.
3. **Rounding:** the standard round-to-nearest-even rule is applied to whatever bits survive alignment, so if the small operand's bits fell outside the guard/round/sticky bits used internally, the rounding step has nothing left to round with.

### Optimization techniques

1. **Switch to double precision:** IEEE 754 double gives 52 mantissa bits and an 11-bit exponent, roughly doubling both the precision and the dynamic range, so a much larger disparity between operand magnitudes can be tolerated before the small value is lost.
2. **Algorithmic compensation – Kahan summation:** when many values of very different magnitude must be summed (typical in scientific/numerical code), Kahan's algorithm keeps a running compensation term that captures the low-order bits lost on each addition and feeds them back into later additions, achieving close to double-precision accuracy even while using single precision storage.
3. **Reordering / pairwise summation:** summing numbers of similar magnitude together first (e.g. sorting by magnitude or using pairwise/tree summation) reduces the exponent gap at each individual addition, which directly reduces how many bits are shifted out at every step.
4. **Scaling:** where the application allows it, rescaling all operands into a comparable numeric range before the computation and rescaling the result afterwards avoids the extreme exponent mismatch altogether.

---

### Code of Conduct:

*I, V Jeevanantham (Reg. No. 192421234), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: \_\_\_\_\_